Plan B

HW2 현실적인 구현 계획 — Amazon Reviews 2023 활용

Amazon Reviews 2023을 **활용**해서 충분히 구현할 수 있어. 다만 데이터셋 전체를 사용하거나 복잡한 추천 모델을 학습시키는 것은 이번 과제에 비효율적이야.

내가 추천하는 방향은 **실제 Amazon 상품 100~200개**로 **작은 쇼핑 데이터베이스**를 만들고, **Information Gain 기반으로 질문을 선택하는 대화형 추천 시스템**을 구현하는 거야.

목표는 세 가지야.

1. **사용자**가 **ChatGPT와 자연스럽게 대화**하면서 **원하는 상품**을 찾을 수 있어야 해.
2. 질문은 무작위가 아니라 **Information Gain에 따라 선택**해야 해.
3. Random, Fixed Question과 비교해서 성능을 정량적으로 평가해야 해.

과제 마감은 2026년 11월 8일 23:59이고, 팀별 API 예산은 $20이야. 따라서 10월 26일부터 11월 7일까지 13일 동안 개발과 평가를 끝내는 것을 목표로 잡자.

1. Amazon Reviews 2023에서 실제로 사용할 데이터

공식 데이터셋은 상품 메타데이터와 사용자 리뷰를 별도로 제공해.
상품 메타데이터에는 상품명, 설명, 특징, 가격, 평점, 이미지 등과 같은 정보가 포함돼 있어.



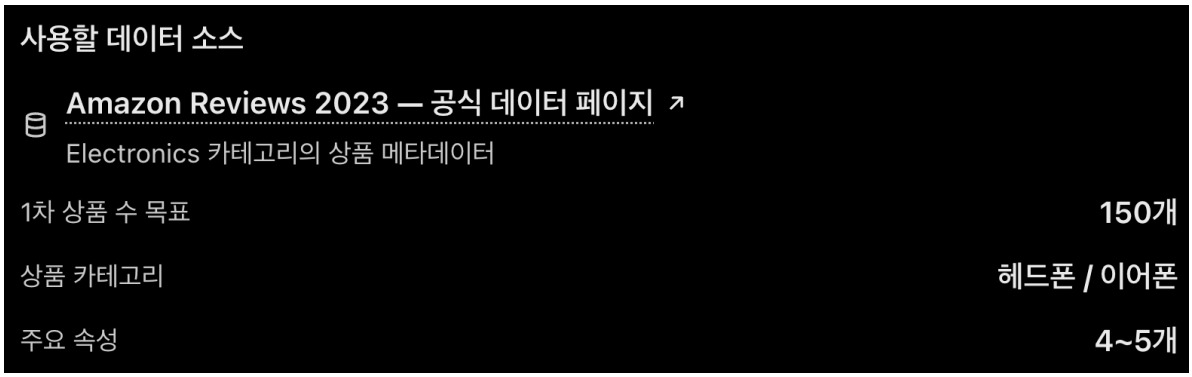
Amazon Reviews'23

우리의 1차 구현에는 리뷰 데이터는 없어도 돼. 상품 메타데이터만 있으면 충분해.

사용할 데이터 소스

Amazon Reviews 2023 — 공식 데이터 페이지

Electronics 카테고리의 상품 메타데이터



상품 데이터는 이런 형태로 만들 거야

Product	Connection	Type	ANC	Price
Headphone A	Wireless	Over-ear	Yes	$150
Headphone B	Wired	Over-ear	No	$60
Headphone C	Wireless	In-ear	Yes	$100
Headphone D	Wireless	In-ear	No	$45

실제 추출 결과가 아닌, 우리가 구축할 데이터 형식의 예시야.

상품마다 `connection`, `type`, `ANC`, `price`를 정리해서 데이터베이스를 만드는 거야.

여기서 중요한 점은 Amazon 원본 데이터에 이 네 가지가 모두 깔끔하게 정리되어 있지는 않다는 거야.

그래서 상품의 `title`, `features`, `details`, `description`에서 필요한 정보를 추출해야 해.

다만 가장 큰 현실적 위험은 상품 속성의 누락이야. 따라서 처음부터 150개를 확정하지 않고, 먼저 30~50개를 샘플링해서 실제 추출 가능성을 확인할 거야.

2. 전체 시스템을 어떻게 구현할까?



상품은 처음부터 150개를 전부 사용자에게 보여주지 않아. 초기 요청으로 카테고리와 기본 조건을 파악한 뒤 후보를 구성하고, 이후 질문을 통해 확률을 갱신할 거야.

대화와 데이터 처리는 별도의 모듈로 만들기 때문에, 실제 Amazon API에 접속하지 않아도 실행 가능한 시스템이 돼.

3. 실제 개발은 5단계로 진행하자

Phase 1. 상품 데이터 확보 및 검증 (10/26~10/28)

목표: 실제 추천에 사용할 수 있는 상품 카탈로그 구축

가장 먼저 공식 데이터 페이지에서 Electronics의 `meta` 파일을 이용하자.

데이터가 매우 크기 때문에 전체를 메모리에 올리지 않고, 압축된 JSONL 파일을 한 줄씩 읽으면서 헤드폰 관련 상품만 필터링할 거야.

구체적으로는 다음 순서로 진행해.

1. **상품명과 카테고리**로 **헤드폰·이어폰을 필터링**한다.
2. 액세서리, 교체 부품, 케이스 등은 제외한다.
3. 상품의 `features`, `details`, `description`에서 연결 방식, 착용 유형, ANC 여부 등을 추출한다.
4. 추출된 속성 중 증거가 없는 정보는 `unknown`으로 저장한다.
5. 속성 누락률, 상품 중복률, 속성별 다양성을 검사한다.

처음에는 30~50개를 수동으로 확인하고, 품질이 충분하면 100~200개로 늘리자.

첫 단계의 통과 기준은 핵심 속성 3개 이상이 신뢰할 수 있게 기록된 상품을 최소 50개 확보하고, 그 속성으로 실제 후보를 구별할 수 있는 것이야.

만약 이 기준에 미달하면 다른 데이터셋을 찾기보다 헤드폰 내에서 더 정보가 풍부한 상품만 선별하거나 카테고리를 바꾸는 것이 현실적이야.

Deliverable: `products.json`

상품 ID, 제목, 원본 설명, 추출 속성, 가격 및 이미지 정보가 들어간 로컬 상품 데이터베이스

Phase 2. Information Gain 엔진 (10/29~10/30)

목표: API 없이 다음 질문을 선택하는 알고리즘 구현

처음에는 실제 상품 20~30개를 하나의 후보 집합으로 만들자.

각 상품에 초기 확률을 부여하고, 가능한 질문의 Information Gain을 계산해.

예를 들어:

질문 후보	예상 IG (가상)
유선 vs 무선	0.92 bits
Over-ear vs In-ear	0.81 bits
ANC 필요 여부	0.55 bits

그러면 첫 번째 질문을 선택하는 거야.

사용자가 "무선"이라고 답하면 무선 상품의 확률을 높이고, **다음 질문에 대한 IG**를 **다시 계산해**.

이 단계에서는 실제 자연어 입력 없이 `wireless`, `over-ear` 같은 구조화된 응답만 사용하면 돼.

구현할 Python 모듈은 세 가지야.

- `belief_tracker.py`: 사용자 선호에 대한 상품별 확률 업데이트
- `question_selector.py`: 각 질문의 IG 계산
- `recommender.py`: 현재 확률에 따른 Top-3 상품 추천

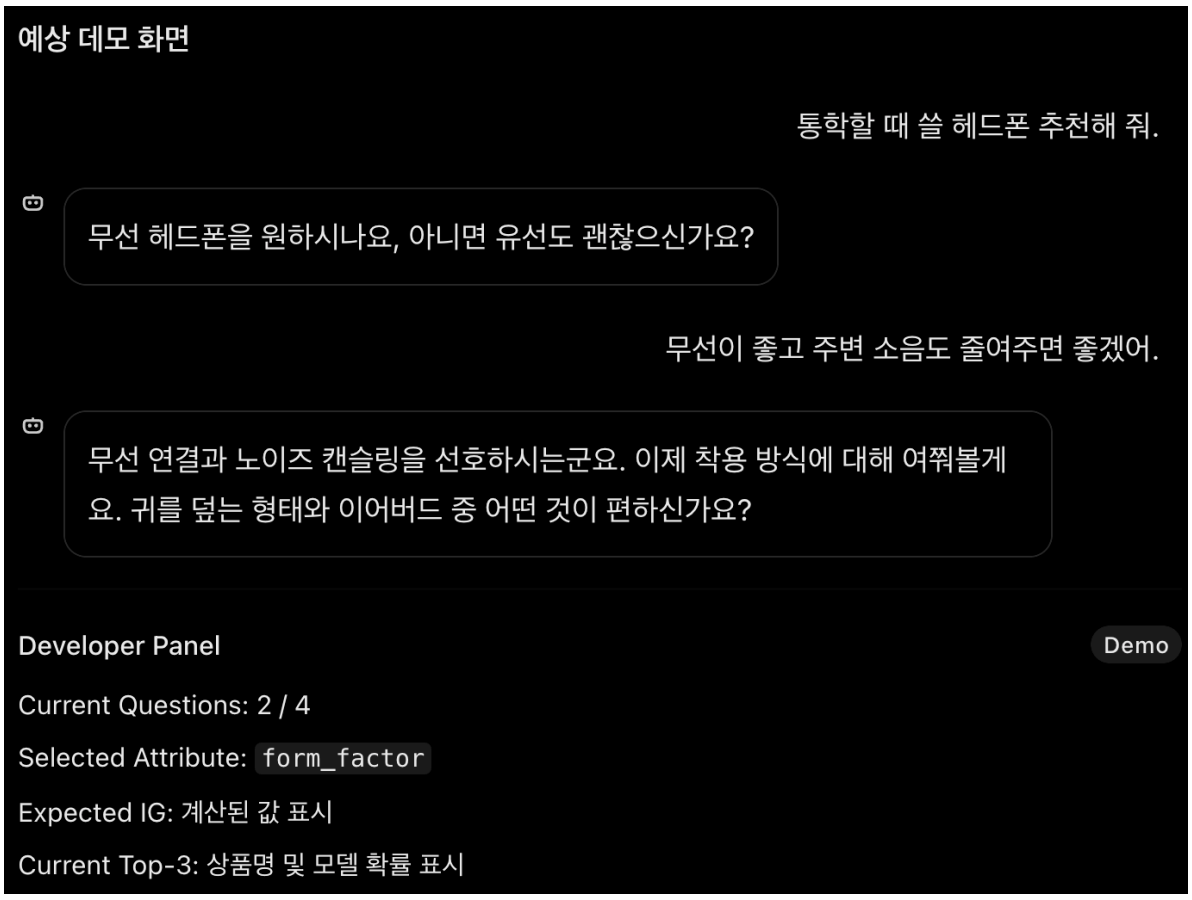
여기서 중요한 설계는 **상품 후보에 대한 초기 확률**이 **균등**하다고 가정하는 **간단한 모델부**터 시작하는 것이야. 이후 초기 검색 결과의 관련성 점수로 사전확률을 설정하는 방법을 추가할 수 있어.

또한 상품에 `unknown` 속성이 많거나 후보들이 같은 속성을 공유하면 질문으로 구별할 수 없으므로, 해당 상황에서는 무리하게 질문하지 않고 추천 근거의 한계를 표시하도록 하자.

Phase 3. OpenAI API 연결 및 챗봇 (10/31~11/2)

목표: 실제 사용자가 자연어로 대화할 수 있는 시스템

여기서는 가벼운 Streamlit 웹 인터페이스를 추천해. 과제에서 화려한 UI를 요구하지 않으므로 웹 페이지 하나면 충분해.



LLM 호출은 최소화하자. 질문 문장 자체는 미리 준비한 템플릿으로 출력해도 되고, OpenAI API는 주로 사용자의 자유로운 답변을 구조화하는 데 사용하면 돼.

여기서 중요한 구현 규칙이 있어. 사용자가 "무선이고 ANC도 필요해"처럼 한 번에 여러 조건을 말했다면 두 조건을 모두 업데이트해야 해. 그렇지 않으면 이미 받은 정보를 다시 질문하는 비효율적인 챗봇이 돼.

또한 사용자가 조건을 정정하면 이전 조건을 무작정 누적하지 않고 해당 속성의 상태를 교체하거나 Belief를 다시 계산하도록 해야 해.

Phase 4. 정량 평가 (11/3~11/5)

목표: Information Gain 방식이 실제로 효과적인지 보여주기

Amazon Reviews 2023은 개인별 선호의 정답을 제공하지 않으므로, 평가에서는 실제 상품을 기반으로 Synthetic User를 만들 거야.

예를 들어 가상 사용자의 숨겨진 조건이 다음과 같다고 하자.

```
{
  "connection": "wireless",
  "form_factor": "over-ear",
  "noise_cancelling": "yes"
}
```

세 알고리즘에 똑같은 조건을 숨겨두고, 각자가 질문하도록 해.

Method	질문 방식
Random	질문을 무작위로 선택
Fixed	정해진 순서로 질문
Information Gain	가장 정보량이 높은 질문 선택

질문할 때마다 시뮬레이터가 숨겨진 조건에 따라 응답하고, 알고리즘은 상품 순위를 업데이트해.

처음에는 30개 정도의 시나리오로 코드를 검증하고, 문제가 없으면 100개까지 늘리자. 질문 예산은 최대 4회로 제한하는 것을 추천해.

핵심 지표는 다음과 같이 설정하자.

Metric	평가 내용
Hit@1	목표 상품을 첫 번째로 추천했는가?
Hit@3	목표 상품이 상위 3개 안에 있는가?
Turn-AUC	적은 질문으로 좋은 성능에 도달하는가?
Constraint Satisfaction	사용자의 명시적인 조건을 충족하는가?

단, 시뮬레이터가 하나의 상품을 목표로 설정하는 방식에서는 속성이 동일한 상품들을 구별하기 어려울 수 있어. 그래서 상품 일치율 외에 선호 조건 충족률도 반드시 평가해야 해.

최대한 공정하게 비교하려면 세 방법 모두 같은 상품, 같은 초기 확률, 같은 사용자 응답 및 같은 추천 알고리즘을 사용하고, 오직 질문 선택 방법만 다르게 해야 해.

그다음 응답 노이즈를 조금 추가해서 IG가 얼마나 안정적으로 작동하는지 확인할 수 있어.

Deliverable: 평가 결과 그래프 2~3개

질문 횟수별 Hit@1, 질문 횟수별 조건 충족률, 응답 노이즈에 따른 성능 비교

Phase 5. Error Analysis, Safety 및 발표 (11/6~11/7)

목표: 5분 안에 Method, Demo, Evaluation을 설득력 있게 제시

최종 발표에서는 다음 순서가 좋아.

시간	내용
0:00–0:40	기존 쇼핑 검색의 한계, Motivation
0:40–1:40	Information Gain 기반 질문 선택 방법
1:40–3:00	실제 대화형 상품 추천 Demo
3:00–4:20	Random/Fixed/IG 성능 비교 및 실패 분석
4:20–5:00	Safety, Limitations, Conclusion

특히 Error Analysis에서는 사용자 발화를 잘못 해석한 경우, 데이터에 없는 상품 속성을 추론한 경우, IG는 높았지만 추천 품질 개선은 적었던 경우를 구분해서 보여주자.

4. 구현 범위를 어디까지 잡아야 할까?

이번 과제에서 꼭 필요한 기능과 과감하게 제외할 기능을 구분하면 다음과 같아.

기능	결정
실제 Amazon 상품 데이터 활용	필수
상품 속성 추출 및 검증	필수
Information Gain 질문 선택	필수
사용자 자연어 이해	필수
Multi-turn Dialogue	필수
Top-3 추천 및 추천 근거	필수
Random/Fixed Baseline	필수
실제 Amazon 실시간 검색 API	제외
구매 및 결제 기능	제외
리뷰 전체를 활용한 모델 학습	제외
Multi-Agent	제외
Multimodal 검색	제외
협업 필터링 모델 학습	제외

이렇게 해야 기능을 충분히 갖추면서도 제출 기한을 맞출 수 있어.

5. 실제 개발 파일 구조

프로젝트를 다음처럼 구성하면 충분해.

```
hw2-shopping-chatbot/
├── data/
│   ├── products.json
│   └── evaluation_profiles.json
├── src/
│   ├── data_processing.py
│   ├── belief_tracker.py
│   ├── question_selector.py
│   ├── recommender.py
│   ├── llm_parser.py
│   └── simulator.py
├── evaluation/
│   ├── baselines.py
│   ├── run_experiments.py
│   └── analyze_results.py
├── app.py
```

```
├── requirements.txt
└── README.md
```

개발 환경은 Python, Pandas, NumPy, OpenAI SDK, Streamlit 정도로 시작하면 되고, 별도의 GPU 서버나 Fine-tuning은 필요하지 않아.

OpenAI API $20 제한에 맞추기 위해 상품 속성 추출은 규칙 기반을 우선 사용하고, 필요한 경우에만 일부 상품에 LLM을 활용하자. 합성 사용자 시뮬레이션과 IG 계산은 전부 로컬 Python으로 수행해 API 비용을 줄일 수 있어.

6. 지금 가장 먼저 할 일

내가 생각하는 첫 번째 Milestone은 챗봇 구현이 아니라 데이터셋 검증이야.

Amazon Reviews 2023에서 실제로 헤드폰 50개를 추출하고 다음 결과를 확인해야 해.

검증 항목	목표
실제 헤드폰 상품 식별	최소 50개
Connection 속성 확보	가능 여부 확인
Form Factor 속성 확보	가능 여부 확인
ANC 속성 확보	가능 여부 확인
가격 결측률	측정 후 사용 여부 결정
속성 조합의 다양성	후보 구분 가능 여부 확인

이 검증이 통과되면 Information Gain 알고리즘은 큰 어려움 없이 구현할 수 있어.

반대로 속성 정보가 부족하다면 상품 수를 늘리는 것보다 속성이 더 명확한 카테고리로 변경하는 것이 훨씬 효과적이야.

최종적으로 내가 추천하는 실행 전략은 다음과 같아.

10/26~28에 실제 데이터 50개를 검증하고, 통과하면 100~200개로 확장한다. 10/30까지 IG 엔진을 완성하고, 11/2까지 OpenAI API를 연결한 챗봇을 만든다. 11/5까지 Baseline 비교와 평가를 끝낸 다음 11/7에 영상을 완성한다.

이 계획은 Amazon 데이터를 이용한 실제 상품 추천이라는 과제의 기능적 요구사항을 충족하면서도, Information Gain 기반 Active Preference Elicitation이라는 명확한 방법론적 기여를 보여줄 수 있어.

가장 중요한 것은 150개 상품이나 높은 정확도 자체가 아니라, 실제 상품 데이터에 근거해 적응적으로 질문하는 시스템을 만들고, 그 질문 방식이 기존 Baseline보다 효과적인지를 검증하는 것이야.